Banking System
A Relational Database Project

B103 Databases & Big Data
Individual Project Report

GISMA University of Applied Sciences

Students: Aleksandr Bogdanov
Student ID: GH1046853

GitHub repository: https://github.com/aIexsystem/SQL-Banking-project
Video demonstration: https://youtu.be/1vDcZo-S7BM

18 June 2026
Database management system: MySQL

1. Introduction

Final project for designing and building a relational database for banking system. I chose the banking domain because it is something that most people understand from everyday life and at the same time it has strong relationship with SQL. The bank system has to keep track of many things simultaneously: where its branches, who its customers, which accounts those customers hold, every transaction that moves money and the loans that people take. The main objective for this project was to create robust database that stores this data without duplication, protects it from bad or impossible values (by using constraints) and still solves real banking questions. To show the design actually works, I also made up a set of SQL queries that cover everyday operations for bank like: adding and updating records, joining tables together, summarising data with aggregations and moving money safely. The whole system was implemented in MySQL and all of the files (schema, sample data, queries and the ER diagram) are available in the GitHub repository linked on the first page.

2. System Design

Before writing whole database, I planned thoroughly the database from the scratch and then drew an entity-relationship diagram (ERD) using Mermaid. The database design has seven tables, each one representing a single clear entity.

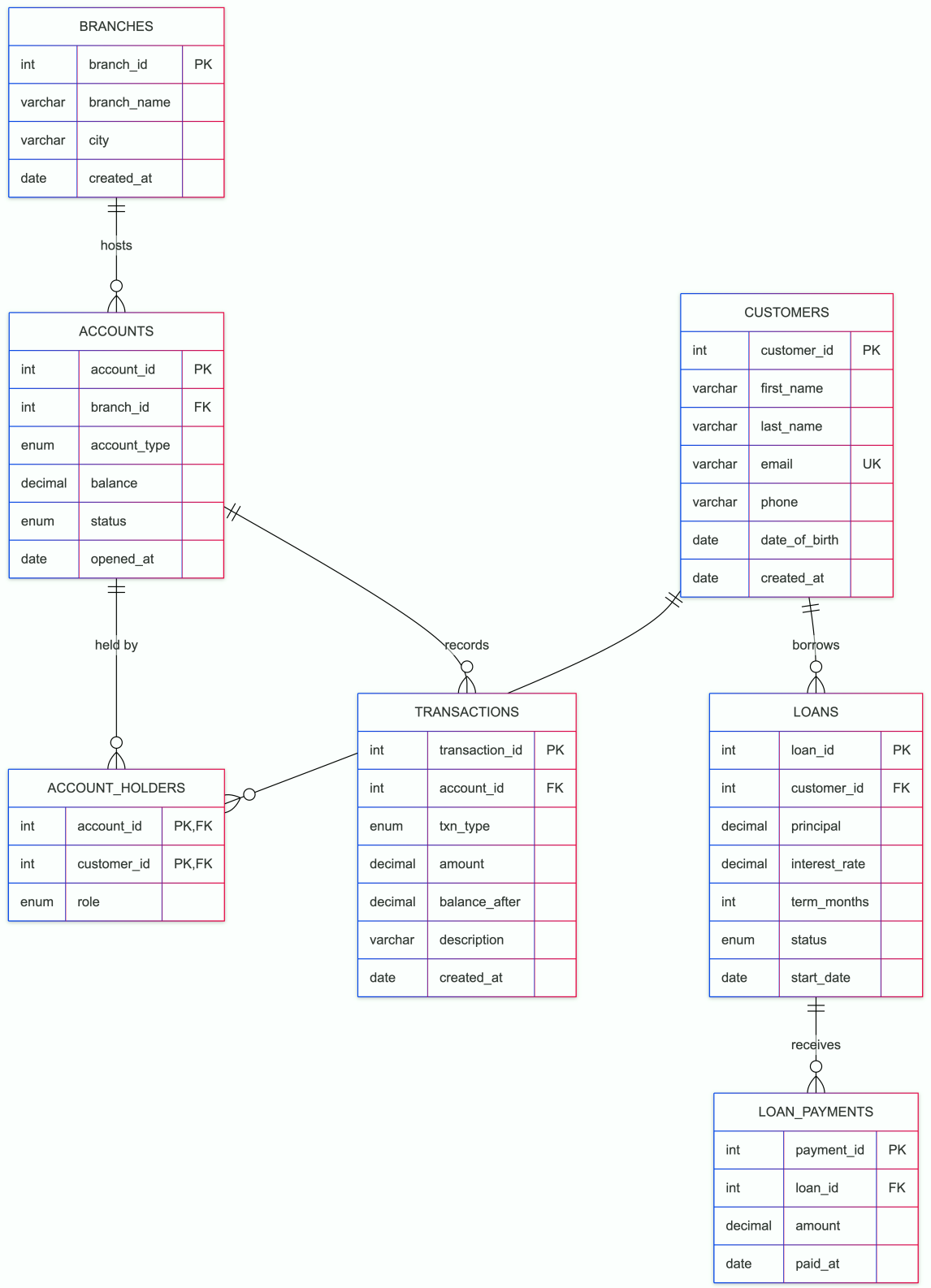
Entities and their attributes

branches store the physical offices of the bank (name and city). **customers** hold personal details such as name, a unique email, phone and date of birth. **accounts** describe each bank account with its type, balance and status. **account_holders** is a junction (link) table that connects customers and accounts. **transactions** record every deposit, withdrawal or transfer. **loans** keep the principal, interest rate and term of each loan and final **loan_payments** list the individual repayments made against a loan.

Relationships

Most of the relationships use are one-to-many. One branch can have many accounts, whereas one account can have many transactions. One customer can have many loans and one loan can have many payments. The most interesting relationship is between customers and accounts. A single customer can hold several accounts, but at the same time the account can also belong to more than one person, for example a joint account shared by a couple. This is a **many-to-many** relationship and the standard way to model it is with a junction table. That is exactly the job of **account_holders**, which uses a composite primary key made of the account id and the customer id plus a role column that marks each person as 'primary' or 'joint'. The ERD below shows all seven tables and how they connect

Figure 1 entity-relationship diagram



ER diagram (drawn in Mermaid.ai)

3. Implementation

First I created the database itself, then the tables in the right order so that every foreign key points to a table that already exists. The code below shows the accounts table, which is a good example of the design choices I made.

```
CREATE TABLE accounts (
   account_id INT AUTO_INCREMENT PRIMARY KEY,
   branch_id INT NOT NULL,
   account_type ENUM('checking','savings') NOT NULL DEFAULT
   'checking', balance DECIMAL(14,2) NOT NULL DEFAULT 0.00,
   status ENUM('active','frozen','closed') NOT NULL DEFAULT 'active',
   opened_at DATE DEFAULT (CURRENT_DATE),
   CONSTRAINT fk_accounts_branch FOREIGN KEY (branch_id) REFERENCES
   branches(branch_id),
   CONSTRAINT chk_balance_nonneg CHECK (balance >= 0)
);
```

Primary and foreign keys

Every table has a primary key. Majority use an AUTO_INCREMENT in id, which is simple and pretty efficient, while account_holders uses a composite key. Foreign keys bond the tables together. For example, you cannot create an account for a branch that does not exist and you cannot record a payment for a loan that was never made as well. This is referential integrity, where database forces it automatically.

Constraints and data types

I tried to push as many of the banking rules as possible down to the database itself, so the data stays clean no matter what. Money is always stored as **DECIMAL(14,2)** instead FLOAT, because floating point numbers cause rounding errors that are unacceptable for money integrity. **CHECK** constraints guarantee that the balance cannot go below zero that a transaction amount and a loan principal are always positive and at the same time loan term is greater than zero. **ENUM** types restrict columns such as account_type, txn_type and status to a fixed list of allowed values, so a typo like 'savigns' is simply rejected. The email column is marked UNIQUE because two customers cannot register with the same email address.

Normalization

The schema is normalized to the 1NF, 2NF and 3NF as well. Each table describes one entity, there are no repeating groups and no column depends on another non key column. For instance, the city is stored only once in the branches table instead of being copied into every account. Only the one deliberate exception is the "balance_after" column in transactions, which stores the running balance at the moment of each transaction. This is a small, conscious denormalization that keeps a useful historical record

Indexing

Primary and foreign keys are indexed automatically, but besides on top of that I added index in the customers' last name, because searching customers by name is a very common operation. Using EXPLAIN, I could see that the same query changed from a full table scan to an indexed lookup after the index was created.

4. Results

To prove that the database meets its goals, I loaded realistic sample data (three branches, six customers, seven accounts, several transactions and three loans) and ran a range of queries, they cover every aspect of handling database to prove it works properly without any deviations : CRUD operations, joins, aggregations, subquerries, safe money transfer and indexing.

Joins and aggregations

Four table join lists every account together with its owner, the owner's role and the branch city. Aggregation queries then summarise the data. for example, the total balance held in each branch returns:

branch_name	num_accounts	total_balance
Berlin Central	3	14,000.00
Munich West	2	3,700.00
Hamburg Port	2	7,500.00

Other aggregations show the average, minimum and maximum balance per account type and the total amount repaid on each loan. "**HAVING**" clause filters the result to branches that hold more than 5000 in total and subqueries find the accounts whose balance is above the overall average, as well as the customers who currently hold a loan.

Safe money transfer (ACID)

The most important and cautious query moves money between two accounts. Transferring money from one account and putting it into another must happen together otherwise it would be a disaster if first step went through successfully and the second one failed. I implemented both updates and the matching transaction records inside a single transaction:

```
START TRANSACTION;
  UPDATE accounts SET balance = balance - 1000 WHERE account_id = 1;
 UPDATE accounts SET balance = balance + 1000 WHERE account_id = 3;
 INSERT INTO transactions (...) VALUES (...);
COMMIT;
```

Because of the ACID properties of the database, the transfer is treated as one single unit of work after commits, account 1 and account 3 both hold 1500.00. I also ran a transfer that was too large on purpose to validate CHECK constraint which refused to let the balance go negative, therefore the statement failed and a ROLLBACK left the balances exactly as they were before transaction. Considering all of that above, these results confirm that the database stores data properly, protects it within set constraints as well as solves real bank problems.

5. Challenges and Solutions

The first real challenge was the joint account problem. My very early design simply put a customer_id column inside the accounts table, but soon realized this could never describe an account shared by two people at once. The solution was the account_holders junction table with a composite key, which models the many-to-many relationship clearly and let record who is primary holder. The second challenge was choosing the right data type for money. At first I used FLOAT, but small tests sum did not add up to the exact cent. After reading about how SQL stores point numbers, I switched every money column to DECIMAL(14,2) which keeps values accurate and precise. Another issue that I struggled with was deciding how much validation should be handled by the DB, despite having the choice to leave all check to the application so I decided to add checks for balance that it cannot go below zero. Furthermore, I also faced issues with the order of CREATE TABLE statements, because the majority of tables depended on each other by foreign key relationships. Solution was to create and define parent tables before children. In general, these mistakes led to the constraint errors and problems with data

6. Conclusion and future work

Overall, the project has achieved its goal: normalized, well-structured banking database that supports realistic operations and answers meaningful business questions. In process of budiling it taught me how schema design, keys, constraints and transactions work together to keep data flexible and reliable. In the future, the database handling could be extended with stored procedures to properly handle transfers. Triggers could be also used to automate logging every change for analyzing purposes.

References

Connolly, T. and Begg, C. (2015) *Database Systems: A Practical Approach to Design, Implementation, and Management. 6th edn. Harlow: Pearson.*

Date, C.J. (2004) *An Introduction to Database Systems.* 8th edn. Boston: Addison-Wesley.

Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems.* 7th edn. Hoboken: Pearson.

Oracle (2024) *MySQL 8.0 Reference Manual.* Available at: https://dev.mysql.com/doc/refman/8.0/en/